

Plataforma Inteligente para Análise e Comparação de Apólices D&O

Relatório Técnico — Projeto Final

Insurminds — extração, organização e comparação de informações de apólices de seguro Directors & Officers com OCR, LLMs e agentes inteligentes

Instituto de Inteligência Artificial Aplicada — I2A2

Curso de Inteligência Artificial Aplicada — Desafio Final

Grupo AlgorithLabs

Integrantes do Grupo:

Giovani Amiti

Pablo Benachio

Eridalgo Ramos da Silva

Outubro de 2026

Protótipo funcional (MVP) — monólito modular em Python

Validação automatizada: 475 testes aprovados

Validação com apólices reais de seguradoras (AXA, Allianz e Porto)

Sumário

Introdução

Problema

Objetivo

Escopo do protótipo

Visão geral da solução

As sete etapas de processamento

Requisitos mínimos do desafio

Arquitetura da solução

Estilo arquitetural: monólito modular

Módulos e responsabilidades

Camadas internas dos módulos

Kernel compartilhado e contratos versionados

Composition root e regras de acoplamento

Tecnologias utilizadas

Agentes e componentes de Inteligência Artificial

Agentes de extração e explicação

Agentes de apoio ao analista

Agentes do Relatório D&O

Guardas de robustez e timeout

Recuperação semântica de evidências

Fluxo completo de processamento

Armazenamento de dados e rastreabilidade

Relatório D&O (metodologia §1–§11)

Interface do usuário

Tratamento de erros

Qualidade, testes e avaliação

Segurança e boas práticas

Justificativa das decisões arquiteturais

Limitações conhecidas

Possibilidades de evolução futura

Instruções de instalação e execução

Conclusão

Anexo A — Matriz de requisitos × evidência

Anexo B — Catálogo de campos e vocabulários

Anexo C — Fontes de dados e referências

1. Introdução

1.1 Problema

Apólices de seguro D&O (Directors and Officers) são documentos longos, complexos e frequentemente redigidos em linguagem jurídica. A comparação entre duas apólices — típica em cotações, renovações e auditorias de placement — exige a identificação manual de cláusulas, coberturas, exclusões, franquias, limites de responsabilidade, vigência, retroatividade e demais condições relevantes.

Esse processo consome várias horas de trabalho de especialistas, é repetido a cada novo documento e, pior, costuma ser entregue **sem rastreabilidade**: a recomendação do analista raramente aponta para a página e o trecho da apólice que a sustentam. Uma exclusão ou um limite inferior em relação à apólice comparada pode passar despercebido — com risco contratual e reputacional para o segurado.

1.2 Objetivo

O objetivo deste projeto foi projetar e desenvolver uma solução baseada em Inteligência Artificial capaz de **extrair, organizar e comparar** informações presentes em apólices de seguro D&O, integrando as tecnologias estudadas ao longo do curso — OCR, LLMs, agentes inteligentes, bancos de dados, automação e interfaces de consulta — para apoiar a tomada de decisão do analista de seguros.

O foco do trabalho não é um produto comercial completo, mas sim **uma arquitetura consistente, bem documentada e tecnicamente fundamentada**, com um protótipo funcional (MVP) que demonstre o fluxo completo: do recebimento do PDF até a apresentação das diferenças entre duas apólices e a geração de um relatório técnico de decisão, com evidência documental anexada a cada informação extraída.

1.3 Escopo do protótipo

- **Entrada:** arquivos PDF de apólices (digitais ou escaneados), um por apólice, em quantidade fixa de duas apólices por comparação.
- **Processamento:** OCR/extração de conteúdo convertido em **markdown estruturado por seções**, indexação vetorial, extração estruturada de 10 campos críticos por agente LLM + achados fora do catálogo, validação e normalização.
- **Saída:** comparação determinística campo a campo, explicações fundamentadas em evidências, agente inteligente de dúvidas, fila de revisão humana, **relatório D&O com ranking de decisão** e exportação em Markdown.
- **Interface:** aplicação web em Streamlit cobrindo a jornada do analista, com launcher de dois cliques e modo de demonstração offline.

2. Visão geral da solução

2.1 As sete etapas de processamento

A plataforma implementa integralmente as sete etapas sugeridas no enunciado do desafio:

#	Etapas	Como é implementada
1	Recebimento dos documentos	Upload de PDFs na interface, com validação de assinatura (%PDF -) e limite de 50 MB por arquivo; o arquivo é salvo temporariamente e registrado como documento da apólice.
2	Extração automática do conteúdo	Texto nativo via PyMuPDF; páginas escaneadas via PaddleOCR; documentos complexos via PP-StructureV3, que devolve markdown estruturado por página (títulos, tabelas e blocos), com cache de predição por página.
3	Organização das informações	O markdown é dividido em seções por títulos (headings), cada seção vira evidência citável com metadados e <i>hash</i> de proveniência; os fatos de negócio são organizados em catálogo fechado e tipado, com normalização por tipo.
4	Armazenamento estruturado	DuckDB (tabelas <code>policies</code> , <code>documents</code> , <code>facts</code> , <code>comparisons</code> , <code>document_markdown</code> , <code>extra_findings</code> , com migrações versionadas) + índice vetorial Qdrant.
5	Consulta das informações	Tela de consulta livre em linguagem natural (busca semântica com citações) + Agente Inteligente de perguntas e respostas + APIs de fatos, evidências, seções de markdown e histórico de revisão.
6	Comparação entre apólices	Motor determinístico A × B campo a campo (IGUAL / MAIOR / MENOR / DIVERGENTE / ausente em um dos lados), com direção e valores normalizados — o LLM jamais compara valores .
7	Apresentação dos resultados	Interface Streamlit com tabela de diferenças, explicação por campo, painel de revisão por mensagem, Relatório D&O com ranking parametrizável e exportação em Markdown.

2.2 Requisitos mínimos do desafio

Todos os requisitos mínimos do enunciado são atendidos. A matriz completa requisito × evidência de código está no **Anexo A**.

Requisito mínimo	Situação	Observação
Ler documentos em PDF ou imagem	Atende	PDF digital e PDF escaneado (imagem interna) via OCR. Imagem avulsa (PNG/JPG) não é aceita diretamente pelo uploader — o pipeline processa imagens convertidas em PDF (ver Limitações).
Extrair automaticamente informações relevantes	Atende	Pipeline automático ponta a ponta; extração de 10 campos críticos + achados fora do catálogo por agentes LLM com guardas de qualidade.
Estruturar os dados extraídos	Atende	Fatos tipados (Pydantic) com status, confiança, evidências e valor normalizado por tipo.
Comparar pelo menos duas apólices	Atende	Fluxo A × B com resultado e direção por campo.
Apresentar as principais diferenças	Atende	Tabela campo × resultado, valores formatados, explicação rastreável e relatório de decisão.
Utilizar IA generativa no processamento	Atende	LLM (Gemini ou modelos via OpenRouter) em 8 tipos de agentes + embeddings para busca semântica.
Interface de demonstração	Atende	Streamlit com a jornada completa + launcher de dois cliques + modo demo offline.

3. Arquitetura da solução

3.1 Estilo arquitetural: monólito modular

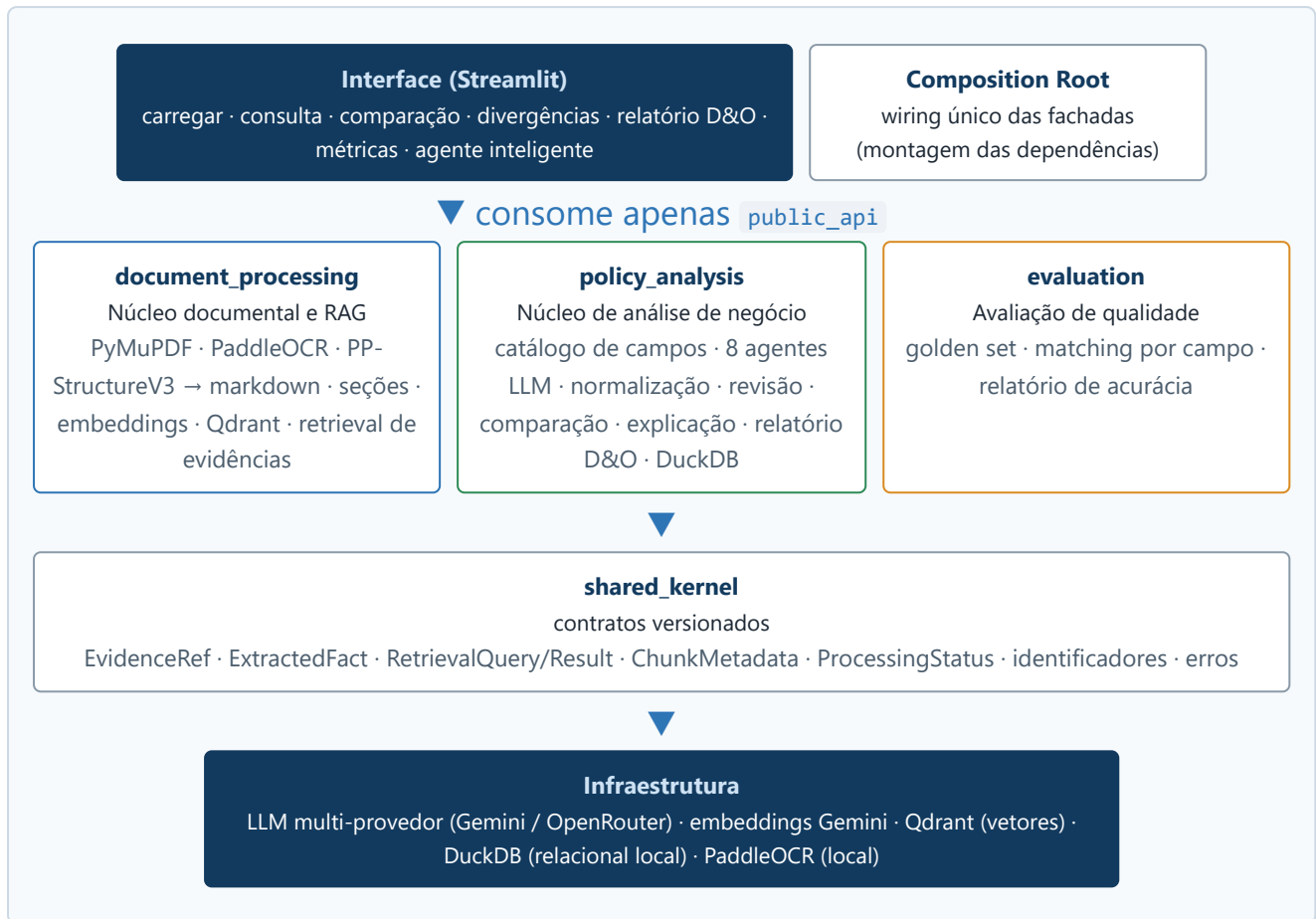
A solução foi construída como um **monólito modular**: um único processo e uma única aplicação Python, com limites internos explícitos, contratos claros e propriedade definida dos dados. A alternativa distribuída (microserviços, filas, brokers) foi rejeitada por princípio: para um MVP single-user, ela adicionaria complexidade operacional sem benefício mensurável — violando os critérios de simplicidade adotados desde o projeto.

Os módulos representam **capacidades de negócio** (e não camadas técnicas como *models* ou *services*), o que impede que qualquer componente importe qualquer outro e evita a evolução para um monólito acoplado.



Figura 1 — Visão geral da arquitetura da plataforma: fluxo ponta a ponta das 7 etapas de processamento, módulos do monólito modular, armazenamento de dados, agentes de IA e tecnologias utilizadas.

3.2 Módulos e responsabilidades



Módulo	Responsabilidade	Entrega de valor
<code>document_processing</code>	Transformar arquivos em evidências recuperáveis: leitura de PDF/imagem, OCR, análise de layout (markdown), segmentação por seções, embeddings, indexação e recuperação.	<code>EvidenceRef</code> — trecho citável (apólice, documento, página, seção, texto, origem e confiança).
<code>policy_analysis</code>	Transformar evidências em fatos e decisões: extração estruturada, achados extras, validação, normalização, revisão humana, comparação determinística, explicação, relatório D&O e exportação.	<code>ExtractedFact</code> , <code>ComparisonResult</code> e Relatório D&O com ranking.
<code>evaluation</code>	Executar o conjunto de referência (golden set) e produzir relatório de acerto por campo.	Métrica objetiva de acurácia da extração.
<code>shared_kernel</code>	Contratos compartilhados, identificadores tipados, convenções de erro e versão de contrato.	Interfaces estáveis entre módulos.
<code>composition_root</code>	Único ponto de <i>wiring</i> : instancia implementações reais e injeta nas fachadas; resolve modelo/provedor via variáveis de ambiente.	Testabilidade (fakes injetáveis) e configuração centralizada.

3.3 Camadas internas dos módulos

Cada módulo segue a mesma estrutura interna, com dependências apontando sempre para dentro (Domain → Application → Infrastructure):

```
src/modules/<módulo>/
├─ domain/          # regras e modelos de negócio puros (sem I/O, sem SDK)
├─ application/     # casos de uso, orquestração e portas (Protocol)
├─ infrastructure/  # adaptadores: SDKs, bancos, OCR, LLM
└─ public_api.py    # fachada pública – única porta de entrada do módulo
```

O domínio não conhece PaddleOCR, PyMuPDF, Qdrant, DuckDB nem o SDK de LLM; conhece apenas portas (interfaces `Protocol`). Os adaptadores de infraestrutura são substituíveis — inclusive por fakes nos testes.

3.4 Kernel compartilhado e contratos versionados

Os modelos que atravessam os limites dos módulos vivem em `shared_kernel/contracts.py`, são validados por Pydantic e versionados (`CONTRACTS_VERSION`):

Contrato	Papel
EvidenceRef	Evidência documental citável: apólice, documento, página, chunk, seção/cláusula, trecho citado, score de recuperação, confiança de OCR e origem (<code>NATIVE_TEXT</code> , <code>PADDLEOCR</code> , <code>PP_STRUCTURE</code>).
ExtractedFact	Fato extraído: campo, status (<code>FOUND</code> , <code>NOT_FOUND</code> , <code>AMBIGUOUS</code> , <code>NEEDS_REVIEW</code>), valor bruto e normalizado, confiança, evidências e sinalização de revisão humana.
RetrievalQuery/Result	Consulta de evidências com filtros (apólice, documento, seção, campo) e resultado rastreável.
ChunkMetadata	Metadados do fragmento indexado: identificadores, página, seção, cláusula, tipo de documento, origem, <i>hash</i> de conteúdo e versão do pipeline.
ProcessingStatus	Estado do processamento por documento, com estágio, progresso e mensagem.

Os identificadores são tipados (`PolicyId` , `DocumentId` , `ChunkId` , `FactId` , `ComparisonId` , `RunId`) para impedir trocas acidentais de contexto — por exemplo, confundir apólice com documento físico. As evoluções recentes (pipeline de markdown, relatório) foram adicionadas como **novas tabelas e fachadas**, sem alterar os contratos versionados — evidência de que o kernel estável cumpriu seu papel.

3.5 Composition root e regras de acoplamento

A regra de ouro da arquitetura é: **um módulo pode chamar a interface pública de outro módulo, mas jamais importar suas classes internas, acessar suas tabelas diretamente ou modificar seus arquivos.** Essa regra é **verificada automaticamente** por testes de arquitetura (`tests/architecture/test_imports.py`), que falham se houver import proibido.

```
# composition_root/root.py – único ponto de wiring do projeto
def build_facades(db_path=..., model_name=None, api_key=None):
    # model_name=None resolve via env LLM_MODEL (ex.: xiaomi/mimo-v2.6-pro
    # no OpenRouter) com fallback gemini-3.8-flash; a chave segue o
    # provedor (OPENROUTER_API_KEY ou GEMINI_API_KEY).
    document = create_default_document_processing()
    policy = create_default_policy_analysis(
        document_processing_facade=document, ...)
    return document, policy
```

A interface gráfica consome exclusivamente as fachadas públicas e o kernel compartilhado — nunca abre conexão com bancos, instancia agentes ou conhece prompts.

4. Tecnologias utilizadas

Tecnologia	Versão/uso	Papel na solução	Justificativa da escolha
Python	3.12 (runtime)	Linguagem do projeto	Ecossistema maduro de IA/OCR, tipagem moderna e integração nativa com todas as bibliotecas do curso.
Pydantic	2.x	Modelos e contratos validados	Validação de fronteira, serialização e contratos versionados entre módulos.
Pydantic AI	<code>pydantic-ai-slim[openai,google]</code>	Framework de agentes (LLM)	Agentes tipados e testáveis, com saída estruturada, <i>retry</i> e suporte a múltiplos provedores (Gemini e OpenRouter) no mesmo código.
LLM multi-provedor	Gemini (<code>gemini-3.x-flash</code>) e OpenRouter (ex.: <code>xiaomi/mimo-v2.6-pro</code>)	IA generativa (extração, explicação, relatório, agentes)	Modelo generativo exigido pelo desafio; seleção via <code>env LLM_MODEL</code> permite trocar provedor/modelo sem mudar código — resiliência e teste de modelos distintos.
Google Gemini Embeddings	<code>gemini-embedding-001</code> (768 dim.)	Vetores das evidências	Busca semântica de trechos; SDK oficial <code>google-genai</code> .
PyMuPDF	<code>fitz</code>	Extração de texto nativo e renderização de páginas	Rápido e preciso em PDFs digitais; base para decidir quais páginas precisam de OCR.
PaddleOCR / PP-StructureV3	<code>paddleocr</code> ≥ 3.7 + PaddlePaddle	OCR de páginas escaneadas e análise de layout → markdown	OCR de alta qualidade, gratuito e local; o PP-Structure devolve o documento em markdown estruturado (títulos, tabelas), preservando a organização do original.
Qdrant	≥ 1.18 (standalone local)	Banco vetorial das evidências	Indexação e busca por similaridade com filtros

			por metadados; execução local simples.
DuckDB	≥ 1.5 (arquivo local)	Persistência relacional (apólices, documentos, fatos, comparações, markdown, achados)	Analytics local em arquivo único, sem servidor; consultas SQL para relatórios e auditoria; migrações versionadas.
Streamlit	≥ 1.65	Interface web do analista	Prototipagem rápida de interfaces de dados em Python, ideal para demonstração do fluxo.
pytest · Ruff · mypy	—	Qualidade contínua	Testes automatizados, lint e checagem estática de tipos executados como <i>gate</i> de qualidade.
Git	—	Controle de versão e colaboração	Dois desenvolvedores em branches separados, com histórico de decisões documentado.

Observação sobre IA generativa. A solução usa IA em três frentes: (1) **LLM** para extração, explicação, relatório e agentes de conversa; (2) **embeddings** para recuperação semântica; (3) **OCR** local para digitalização. O texto das apólices enviado ao LLM é uma decisão explícita e registrada (ver seção 12).

5. Agentes e componentes de Inteligência Artificial

A solução desenvolve **agentes especializados para tarefas distintas**, conforme a boa prática sugerida no enunciado. Cada agente tem responsabilidade única, contrato de entrada/saída tipado e **prompt versionado com *fingerprint*** para auditoria.

5.1 Agentes de extração e explicação

Agente	Prompt	Responsabilidade
MultiFieldExtractionAgent	extract-v1	Transforma evidências em fatos tipados dos 10 campos do catálogo. Agente multi-campo : uma única chamada de LLM por apólice (reduz custo/latência), devolvendo um fato independente por campo, com evidências e status.
LLMExplanationAgent	explain-v1	Explica em linguagem de analista a diferença calculada entre as apólices, ancorado nas evidências dos dois lados. O agente explica a diferença, não a decide .

5.2 Agentes de apoio ao analista

Agente	Prompt	Responsabilidade
LLMQuestionAgent	question-v1	Agente Inteligente de perguntas e respostas sobre as apólices: RAG com citações (a resposta aponta as evidências usadas), com controles de busca ocultos na UI.
LLMReviewAgent	review-v1	Revisão por mensagem : o analista escreve a correção em linguagem natural ("o limite é X") e o agente aplica a correção estruturada, registrando <code>decided_by = "Analista via Agente IA"</code> .
LLMExtraFindingsAgent	extras-v1	Achados fora do catálogo : registra informações relevantes não previstas nos 10 campos em <code>extra_findings</code> , exibidas à parte na UI — modelo híbrido que preserva o catálogo fechado sem perder descobertas.
InsurerNameAgent	insurer-v2	Identifica a seguradora real do documento (com fallback do usuário) para exibição de rótulos fiéis ao mercado.

5.3 Agentes do Relatório D&O

O relatório de decisão (seção 8) é construído por um conjunto de agentes com prompts versionados (`identify`, `fill`, `tables`, `scenarios`, `scores`, `conclusions`, todos `*-v1`) orquestrados em **map/reduce por categoria**: cada categoria tem sua própria chamada de LLM — nunca uma chamada gigante — e o resultado é consolidado deterministicamente pela matemática pura do domínio.

5.4 Guardas de robustez e timeout

Saídas de LLM são tratadas como **não confiáveis por definição**. Camadas de defesa implementadas e cobertas por testes:

Camada	Problema tratado
Normalização de saída	Remove cercas de Markdown e desembrulha JSON serializado como string antes da validação.
Validação estrutural	Formato inválido vira erro classificado (<code>LLM_SCHEMA_INVALID</code>), nunca dado corrompido.
<code>normalize_value</code>	Converte o valor bruto no valor tipado do campo; forma irreconhecível levanta erro de normalização → fato <code>NEEDS_REVIEW</code> (revisão humana), nunca exceção vazando.
Timeout de LLM (300s)	Conexão presa no provedor degrada para <code>LLM_TIMEOUT</code> reexecutável — antes, uma conexão pendurada bloqueava o fluxo para sempre.
Guarda de rastreabilidade (§10)	No relatório, célula sem referência de evidência vira "Requer confirmação" — dado sem fonte nunca é apresentado como fato.

5.5 Recuperação semântica de evidências

O documento é convertido em **markdown estruturado por página** e dividido em seções por títulos; cada seção vira `EvidenceRef` citável (IDs determinísticos como `ev_md_{apólice}:p{n}:s{i}` — nunca inventados), com sobreposição léxica por campo e truncamento seguro. As evidências recebem embeddings vetoriais; a consulta em linguagem natural é convertida em vetor e buscada no Qdrant por similaridade, com filtros por apólice/documento.

5.6 Componentes de fixture (testabilidade)

Agentes de fixture (`FixtureExtractionAgent` , `FixtureExplanationAgent` e demais fakes) devolvem saídas prontas para testes e modo demo. Eles trocam apenas o adaptador de infraestrutura — os casos de uso são os mesmos do modo real.

6. Fluxo completo de processamento



Detalhamento por estágio

- 1. Recebimento.** A interface grava o upload em área temporária e registra o documento associado à apólice. Arquivos inválidos (assinatura diferente de `%PDF-` ou maiores que 50 MB) são rejeitados na fronteira.
- 2. Extração de conteúdo.** Texto nativo quando disponível; páginas sem camada de texto são renderizadas e enviadas ao OCR. O status publica o estágio (`RECEIVED` → `TEXT_EXTRACTED` → `OCR_COMPLETED`) com progresso visível.
- 3. Organização em markdown.** O PP-StructureV3 devolve markdown estruturado por página (com **cache de predição por página** — regiões e markdown compartilham uma única rodada do engine); o markdown é dividido em seções por títulos, cada seção com metadados e *hash* de proveniência, e cacheado em banco por documento (`document_markdown`), de modo que reproprocessamentos e prévias não repetem OCR.
- 4. Armazenamento estruturado.** Seções vetorizadas e indexadas no Qdrant; metadados e markdown no DuckDB (estágio `INDEXED`).
- 5. Recuperação.** Para cada campo do catálogo, evidências candidatas são recuperadas por sobreposição léxica + busca semântica, preservando página, seção e trecho.
- 6. Extração estruturada.** O agente multi-campo devolve os fatos campo a campo; valores são normalizados e validados. Em paralelo, o agente de achados registra informações relevantes fora do catálogo (idempotente por delete+insert).
- 7. Validação e revisão.** Fatos sinalizados vão para a fila de revisão; o analista corrige por mensagem (agente aplica) ou diretamente — tudo registrado.
- 8. Comparação.** O motor determinístico compara os valores normalizados das duas apólices e produz o resultado (`IGUAL` , `MAIOR` , `MENOR` , `DIVERGENTE` , `AUSENTE_A` , `AUSENTE_B` , `AUSENTES_AMBOS`) com direção.

9. **Explicação.** Para cada diferença, o agente de explicação justifica o resultado com as evidências dos dois lados.
10. **Relatório D&O e exportação.** O relatório de decisão é gerado por categorias e exportado em Markdown standalone; a interface mostra as diferenças, o agente inteligente responde dúvidas e as métricas fecham a auditoria.

7. Armazenamento de dados e rastreabilidade

7.1 Persistência relacional (DuckDB)

Um banco DuckDB local (`exports/policy_analysis.duckdb`) guarda os dados de negócio, com migrações versionadas (baseline + migração 2):

Tabela	Conteúdo
<code>policies</code>	Apólices analisadas (identificador lógico, rótulos de exibição pela seguradora real).
<code>documents</code>	Documentos físicos associados a cada apólice.
<code>facts</code>	Fatos extraídos por campo: status, valor, valor normalizado, confiança, evidências e decisões de revisão.
<code>comparisons</code>	Comparações calculadas $A \times B$, com resultados por campo e explicações.
<code>document_markdown</code>	Cache do markdown estruturado por documento (por <i>fingerprint</i>) — evita reprocessar OCR.
<code>extra_findings</code>	Achados relevantes fora do catálogo de 10 campos (idempotente).

7.2 Índice vetorial (Qdrant)

As seções de markdown com embeddings ficam no Qdrant (instância local), com *payload* de metadados que permite filtrar por apólice, documento, página e seção. A busca é feita exclusivamente pelo módulo documental — a análise nunca acessa o banco vetorial diretamente.

7.3 Rastreabilidade e proveniência

- **Proveniência por *hash*:** documento e seções carregam SHA-256 do conteúdo — detecção de duplicatas e rastreio de reprocessamentos.
- **Evidência obrigatória:** todo fato e toda célula do relatório apontam para evidências (apólice, documento, página, seção, trecho) — dado sem fonte é rotulado "Requer confirmação".
- **IDs determinísticos:** as evidências de markdown têm IDs derivados do documento (`ev_md_{apólice}:p{n}:s{i}`), consistentes entre DuckDB e Qdrant.
- **Auditoria de LLM:** cada chamada registra modelo, versão de prompt, *fingerprint* do prompt, tokens e custo estimado.
- **Origem do texto:** cada evidência declara se veio de texto nativo, OCR ou análise de layout — e a confiança associada.

8. Relatório D&O (metodologia §1–§11)

Diferencial da solução: além da comparação campo a campo, a plataforma gera um **relatório técnico de decisão** com metodologia explícita em 11 seções — da identificação das seguradoras ao checklist final de verificação.

8.1 Vocabulário fechado e catálogo do relatório

O relatório usa **vocabulário controlado**: status de cobertura com "Não localizado" (nunca "não existe" — a ausência na busca não prova a inexistência contratual), níveis de importância e ~80 campos organizados em 8 categorias:

Categoria	Exemplos de conteúdo
Estrutura da proteção	Quem é segurado, quais entidades, escopo da cobertura.
Gatilho temporal	Claims-made, retroatividade, notificação de circunstâncias.
Limites e exposição financeira	LMG, LMIs, sublimites, agregados.
Custos de defesa e regulação	Defesa dentro/fora do limite, custos administrativos.
Coberturas e extensões	Extensões contratadas, coberturas especiais.
Exclusões	Exclusões padrão e específicas por seguradora.
Defesa, separabilidade e conflitos	Separabilidade de interesses, conflito de posições.
Procedimentos e obrigações	Notificação, cooperação, mudança de risco.

8.2 Geração, ranking e sensibilidade

- **Map/reduce por categoria**: cada categoria é preenchida por uma chamada de LLM dedicada (nunca uma chamada gigante), e o resultado é consolidado deterministicamente.
- **Ranking parametrizável (§8)**: nota final por apólice = $\Sigma(\text{nota} \times \text{peso}) / 10$, com **painel de pesos interativo** na interface — o analista ajusta a importância de cada critério e vê o ranking recalculado.
- **Cenários de sensibilidade (§8.3)**: o relatório mostra como o ranking se comporta sob diferentes perfis de prioridade (custo, amplitude de cobertura, rigidez de exclusões).
- **Guarda anti-falso-positivo (§9)**: sem prêmio, LMG, LMIs, franquias, sublimites e perfil de decisão, o relatório se recusa a declarar "vencedora geral" — dado incompleto não vira recomendação.
- **Checklist de verificação final (§11)**: 14 itens de conferência humana antes de fechar a análise.
- **Ressalva fixa (§7)**: os cenários sempre exibem a ressalva metodológica correspondente.

8.3 Exportação

O export gera um **Markdown standalone**: matriz comparativa (§5), 5 tabelas analíticas (§6), ranking + sensibilidade (§8), conclusões (§9) e checklist (§11) — documento único para anexar à recomendação ao cliente.

9. Interface do usuário

A interface (Streamlit) organiza a jornada do analista em seções numeradas, consumindo somente as fachadas públicas:

Seção	Função
1. Carregar as apólices	Envio dos PDFs A e B, com validação e rótulos pela seguradora real detectada.
2. Consulta livre às apólices	Pergunta em linguagem natural; respostas em tabela de evidências com filtros por apólice.
3. Comparação	Tabela campo × resultado (IGUAL/MAIOR/MENOR/DIVERGENTE/ausente), valores formatados, direção, explicação por campo e "Campos adicionais identificados pela LLM".
4. Alguma divergência? Me indique por aqui:	Painel de revisão com severidade; o analista corrige por mensagem e o agente aplica a correção.
5. Relatório D&O	Painel de pesos interativo do ranking (§8), geração, sensibilidade, checklist (§11) e exportação.
Métricas	Qualidade da extração/comparação e métricas de uso do LLM (tokens, custo estimado, modelo).
Agente Inteligente	Barra "Digite sua dúvida" com respostas em linguagem natural citando as evidências usadas.

A aplicação conta com **launcher de dois cliques** (`iniciar_app.bat`) para a demonstração.

10. Tratamento de erros

Erros são **classificados** (nunca texto livre solto) e tratados no nível adequado:

Situação	Tratamento
Arquivo inválido (não-PDF, > 50 MB)	Rejeição na fronteira, antes do processamento, com mensagem objetiva na interface.
Falha de OCR/extração em página	Estágio <code>FAILED</code> com mensagem; processamento reexecutável; falhas parciais identificáveis.
Indisponibilidade do LLM / SDK ausente	Erro classificado (<code>LLM_CLIENT_UNAVAILABLE</code>) — a aplicação não trava; os testes seguem com fakes.
Timeout da API de LLM (300s)	Degrada para <code>LLM_TIMEOUT</code> reexecutável — conexão pendurada no provedor não bloqueia mais o fluxo.
Resposta do LLM fora do schema	Normalização da saída; persistindo inválido, erro <code>LLM_SCHEMA_INVALID</code> — dado corrompido jamais é salvo.
Valor não normalizável (EC-04)	Fato marcado <code>NEEDS_REVIEW</code> e enviado à revisão humana.
Campo ausente na apólice	Status <code>NOT_FOUND</code> explícito, resultado de comparação <code>AUSENTE_A/AUSENTE_B</code> e vocabulário "Não localizado" no relatório — ausência é informação, não erro.
Erro interno inesperado	Exibição de mensagem clara na UI; detalhe completo registrado no log do servidor.

11. Qualidade, testes e avaliação

11.1 Gate de qualidade

Toda integração passa por um *gate* automatizado executado localmente: **lint (Ruff)** + **checagem de tipos (mypy)** + **suíte de testes (pytest)**. Nenhuma entrega é considerada pronta sem o gate verde.

11.2 Suíte de testes

475 testes aprovados, 3 pulados (os pulados exigem dependências externas). As camadas de teste cobrem:

- **Contratos e arquitetura:** validação dos modelos compartilhados e proibição de importações fora da `public_api`.
- **Domínio:** normalização, comparação determinística, matemática do relatório (ranking, sensibilidade, guarda §9) e vocabulários.
- **Aplicação:** casos de uso com fakes (sem rede), falhas de LLM, timeout, respostas fora do schema, fila de revisão e achados extras.
- **Integração:** pipeline documental com PDFs reais de *fixture* (inclusive escaneado), markdown por seções, índice vetorial e migrações.
- **Interface:** componentes da UI com fachadas falsas (rótulos, métricas, consulta, sanitização de erros).

11.3 Validação com apólices reais

A bancada `tests/apolices/validate_real.py` executa rodadas de 2 arquivos sobre **apólices reais de seguradoras** (AXA, Allianz e Porto), gerando JSON de saída por rodada. Resultados registrados na rodada 1 (AXA + Allianz 2025):

Verificação	Resultado observado
Detecção da seguradora	"AXA Seguros" e "Allianz Seguros" identificadas corretamente (leitura direta).
Conversão em markdown	104 e 75 páginas processadas (~250 mil caracteres cada), com seções estruturadas.
Achados fora do catálogo	12 e 17 achados relevantes registrados — cobertura além dos 10 campos.
Extração do catálogo	2/10 e 1/10 FOUND — coerente: os PDFs são Condições Gerais, e valores como LMG/franquia residem na especificação (ver Limitações).
Relatório D&O	Gerado com 23 chamadas de LLM sequenciais — latência identificada como pendência de otimização.

11.4 Avaliação de acurácia e observabilidade

Um conjunto de referência (*golden set*) é executado contra o pipeline e pontuado por regras determinísticas de *matching* por campo, produzindo relatório de acerto por campo — permite medir

regressões quando prompts ou modelos mudam. Cada chamada de LLM registra modelo, versão de prompt, *fingerprint*, tokens e custo estimado.

12. Segurança e boas práticas

- **Credenciais ocultas:** as chaves de API (Gemini e/ou OpenRouter) são lidas exclusivamente de variáveis de ambiente (`GEMINI_API_KEY` , `OPENROUTER_API_KEY`) em tempo de execução. Nenhuma chave existe no código, em arquivos de configuração ou no histórico do repositório Git — verificado por varredura.
- **Arquivos tratados como não confiáveis:** validação de assinatura e tamanho antes do processamento; uploads ficam em área temporária e são removidos após o fluxo.
- **Texto de apólice como dado não confiável:** a saída do LLM é sempre validada por schema antes de virar dado de negócio.
- **Privilegio mínimo do LLM:** os modelos apenas geram texto estruturado — não têm acesso a escrita em bancos, arquivos ou comandos.
- **Dados sensíveis:** apólices são documentos confidenciais. O envio de trechos à API de LLM é uma decisão explícita e consciente registrada no projeto; OCR e processamento pesado rodam localmente. As apólices de teste do repositório são versões de teste dos documentos.
- **Erros com mensagens claras:** a UI exibe erros classificados e compreensíveis, com detalhe técnico no log do servidor.

13. Justificativa das decisões arquiteturais

Decisão	Alternativa considerada	Justificativa
Monólito modular (um processo)	Microserviços, filas/brokers	MVP single-user: complexidade operacional distribuída sem benefício mensurável. Limites internos rigorosos entregam o mesmo ganho de manutenção sem o custo de distribuição.
Comparação 100% determinística	LLM comparando as apólices ou similaridade vetorial	Comparar valores é tarefa de regra, não de geração: determinismo, auditabilidade e eliminação do risco de alucinação no resultado final. O LLM extrai e explica — nunca decide qual apólice é melhor.
Evidência obrigatória em todo fato	Fatos sem fonte (apenas o valor)	Rastreabilidade para o analista defender a recomendação e principal barreira contra alucinação: todo valor aponta página e trecho verificáveis. No relatório, célula sem referência vira "Requer confirmação".
Markdown como formato intermediário	Blocos soltos de OCR / texto corrido	O PP-Structure preserva a estrutura do documento (títulos, tabelas) em markdown; seções por título viram unidades citáveis de evidência — recuperação por RAG muito mais precisa que texto corrido.
Modelo híbrido de campos	Só o catálogo fechado / extração totalmente livre	Os 10 campos críticos preservam comparação determinística e contratos; os achados extras capturam o que o catálogo não previu, sem contaminar a comparação.
Extração multi-campo em 1 chamada por apólice	1 chamada de LLM por campo	Redução de ~10× em chamadas, custo e latência; o schema de saída mantém um fato independente por campo.
Relatório em map/reduce por categoria	Uma única chamada gigante com o documento inteiro	Prompts menores e mais precisos por categoria, limites de contexto respeitados, falha isolada por categoria e auditoria por etapa.
Ranking com pesos do usuário + guarda §9	Score fixo definido pelo sistema	O valor da análise depende do perfil do cliente (custo × cobertura); pesos parametrizáveis tornam o ranking útil. A guarda impede recomendação com dados incompletos.
Multi-provedor via env (LLM_MODEL)	Provedor único fixo no código	Trocar Gemini ↔ OpenRouter (e modelos locais) sem mudar código: resiliência a indisponibilidade, comparação de modelos e controle de custo.
Timeout de LLM de 300s	Espera indefinida / timeout curto	Conexões penduradas no provedor bloqueavam o fluxo para sempre; com teto, a falha é classificada e reexecutável.

Cache de predição de OCR e de markdown	Reprocessar a cada exibição	OCR é a etapa mais cara; regiões e markdown compartilham uma rodada do engine e o resultado é cacheado por <i>fingerprint</i> — prévias e reprocessamentos não repetem OCR.
Contratos compartilhados definidos antes do código	Cada módulo com seus próprios modelos	Dois desenvolvedores em paralelo: contratos estáveis evitam retrabalho — e as evoluções recentes foram adicionadas sem tocar o kernel versionado.
Acesso entre módulos só via <code>public_api</code> , verificado por teste	Convenção não verificada	Regra de arquitetura imposta por teste automatizado não degrada com o tempo.
DuckDB + Qdrant locais	PostgreSQL + pgvector, banco vetorial gerenciado	Execução local, arquivo único, sem servidor e sem custo recorrente; suficiente para o volume do protótipo.
OCR local (PaddleOCR/PP-Structure)	OCR gerenciado (Textract, Vision, Document Intelligence)	Sem custo por página e sem expor o conteúdo confidencial das apólices na etapa de OCR.
Revisão humana como etapa do fluxo	Confiança cega no LLM	Casos ambíguos precisam de especialista; registrar a decisão humana (inclusive via agente) mantém auditoria.
Prompts versionados com <i>fingerprint</i>	Prompts espalhados no código	Mudança de prompt muda comportamento: versionar permite rastrear regressões medidas pelo módulo de avaliação.

14. Limitações conhecidas

1. **Imagem avulsa não aceita no upload:** o uploader aceita apenas PDF; PDF escaneado (imagem interna) é processado via OCR e imagens são suportadas pelo pipeline quando convertidas (os testes incluem um JPG de apólice convertido), mas o envio direto de PNG/JPG ainda não foi liberado na interface.
2. **Comparação restrita a duas apólices:** o fluxo é fixo $A \times B$ (atende o mínimo do desafio); não há comparação de três ou mais apólices nem de versões históricas em lote.
3. **Extração do catálogo em Condições Gerais:** em apólices reais compostas apenas pelas condições gerais, a extração dos 10 campos encontra poucos valores (2/10 e 1/10 FOUND na validação) — os valores (LMG, franquias) residem na especificação, que precisa ser anexada. Pendente de validação com apólice completa.
4. **Latência do Relatório D&O:** a geração por categorias usa ~23 chamadas sequenciais de LLM (40–90 min em modelos mais lentos); otimização por lote/paralelização é a principal pendência de performance.
5. **Campos limitados ao catálogo:** 10 campos críticos para a comparação determinística (achados extras são informativos, não comparáveis) — decisão de escopo para garantir qualidade.
6. **Dependência de API externa de LLM:** a extração e o relatório dependem da disponibilidade/custo do provedor; o multi-provedor mitigou, mas não eliminou, a dependência (não há modelo local 100% offline).
7. **Qualidade do OCR:** PDFs escaneados com baixa resolução ou ruído reduzem a confiança das evidências — mitigado (não eliminado) pela sinalização para revisão humana.
8. **Validação real parcial:** rodada 1 concluída (AXA + Allianz 2025); rodadas com Porto e Allianz 2017 (JPG) ainda pendentes.
9. **Ambiente de demonstração:** a demonstração "real" exige chave de API, Qdrant local e bibliotecas pesadas (OCR); sem isso, roda apenas o modo demo offline com dados de fixture.
10. **Single-user:** não há autenticação, autorização nem múltiplos usuários simultâneos.

15. Possibilidades de evolução futura

Evolução	Benefício esperado
Otimizar o Relatório D&O (agrupar categorias por chamada / paralelizar)	Reduzir de ~23 chamadas sequenciais para poucas chamadas em lote — ganho direto de latência e custo.
Validação com apólice completa (condições gerais + especificação)	Elevar a cobertura da extração dos 10 campos em documentos reais, onde os valores moram.
Upload de imagem avulsa (PNG/JPG)	Atende integralmente a leitura "PDF ou imagem" do enunciado; reutiliza o OCR já existente.
Comparação de N apólices e auditoria de renovação	Cenário real de renovação: comparar a apólice nova com a anterior e destacar mudanças automaticamente.
Modelo local (ex.: via Ollama) no multi-provedor	Operação 100% offline e sem custo por token para documentos sensíveis.
Expansão do catálogo de campos e de exclusões por seguradora	Cobertura de cláusulas específicas do mercado brasileiro e comparação contra padrões SUSEP.
Avaliação avançada (LLM-as-judge, benchmark de acurácia)	Medição contínua da qualidade da extração entre versões de prompt/modelo.
Relatório final em PDF e dashboard	Entrega pronta ao cliente, com gráficos de diferenças e evolução de custo por análise.
Multiusuário, autenticação e auditoria completa	Uso corporativo com trilha de auditoria por usuário.

16. Instruções de instalação e execução

16.1 Pré-requisitos

- Python 3.12 (recomendado; mínimo 3.11) e `pip`.
- Qdrant local na porta `6333` — executável standalone ou `docker run -p 6333:6333 qdrant/qdrant`.
- Chave de API de LLM (Gemini e/ou OpenRouter) para o modo real.

16.2 Instalação

```
git clone <url-do-repositório> Insurminds---Projeto-Final-OCR
cd Insurminds---Projeto-Final-OCR
python -m venv .venv
.venv\Scripts\activate          # Linux/macOS: source .venv/bin/activate
pip install -e ".[dev]"         # stack completo do projeto (instalável)
```

16.3 Configuração de credenciais (nunca no código)

```
# Windows (PowerShell) – apenas para a sessão atual:
$env:GEMINI_API_KEY = "sua-chave"          # provedor Gemini
$env:OPENROUTER_API_KEY = "sua-chave"     # provedor OpenRouter (opcional)
$env:LLM_MODEL = "xiaomi/mimo-v2.6-pro"   # modelo via OpenRouter (opcional;
                                           # fallback: gemini-3.8-flash)

# Linux/macOS: export GEMINI_API_KEY="sua-chave" (etc.)
```

Segurança: nunca grave as chaves em arquivos do repositório nem as commit no Git. A aplicação lê as variáveis de ambiente em tempo de execução.

16.4 Execução

```
# Pelo launcher de dois cliques (Windows):
iniciar_app.bat

# Ou manualmente – interface completa (modo real):
.venv\Scripts\python.exe -m streamlit run src/ui/app.py

# Modo demonstração offline (sem LLM/OCR/Qdrant – usa fixtures):
.venv\Scripts\python.exe -m streamlit run run_ui_demo.py
# (com LLM_REAL=1 o mesmo demo usa as fachadas reais)

# Gate de qualidade:
.venv\Scripts\python.exe -m pytest -q    # testes
ruff check .                             # lint
mypy src                                 # tipos
```

No modo demo, use os identificadores de apólice POL-A e POL-B para percorrer toda a jornada (extração, consulta, comparação, explicação, revisão, relatório e export).

17. Conclusão

O protótipo entrega o fluxo completo solicitado no desafio — recebimento, extração automática, organização, armazenamento estruturado, consulta, comparação e apresentação — com uma arquitetura modular verificada por testes (475 testes automatizados) e decisões técnicas justificadas.

Os diferenciais da solução estão em quatro pontos: (1) **rastreabilidade total** — todo fato e toda célula do relatório apontam para a página e o trecho da apólice que os originam; (2) **comparação determinística** — o LLM extrai e explica, mas nunca decide valores, eliminando alucinação no resultado apresentado; (3) **decisão apoiada** — o Relatório D&O com ranking parametrizável, cenários de sensibilidade e checklist transforma a comparação em recomendação defensável; e (4) **qualidade de engenharia** — contratos versionados, regras de acoplamento impostas por teste, erros classificados com timeout, revisão humana no fluxo e validação com apólices reais de seguradoras (AXA, Allianz e Porto).

As limitações conhecidas — aceitação restrita a PDF no upload, dependência de API de LLM, latência do relatório e necessidade de apólice completa (com especificação) para cobertura total dos campos — estão mapeadas, mitigadas onde possível e documentadas junto com as evoluções previstas. A arquitetura foi desenhada para absorver essas evoluções sem refatoração dos módulos centrais: o multi-provedor, o cache de markdown e o modelo híbrido de campos já são evidências dessa adaptabilidade.

Anexo A — Matriz de requisitos × evidência

A.1 As sete etapas

#	Etapas	Situação	Evidência no código
1	Recebimento dos documentos	Atende	src/ui/components/upload.py · validação em src/modules/document_processing/application/service.py
2	Extração automática do conteúdo	Atende	document_processing/infrastructure/extractors.py , layout.py (PP-StructureV3 → markdown)
3	Organização das informações	Atende	domain/structure.py (seções), policy_analysis/infrastructure/markdown_source.py , domain/field_catalog.py , value_types.py
4	Armazenamento estruturado	Atende	policy_analysis/infrastructure/duckdb_repository.py · document_processing/infrastructure/indexing.py
5	Consulta das informações	Atende	src/ui/components/query.py · agente LLMQuestionAgent · APIs de fatos/evidências/seções
6	Comparação entre apólices	Atende	policy_analysis/domain/comparison.py
7	Apresentação dos resultados	Atende	src/ui/app.py , components/report.py · infrastructure/report_export.py

A.2 Requisitos mínimos

Requisito	Situação	Evidência
Leitura de PDF ou imagem	Atende (com ressalva)	PDF digital e escaneado via OCR; imagem avulsa via conversão (upload PDF-only — ver Limitações).
Extração automática de informações	Atende	infrastructure/llm_agent.py (agentes multi-campo, extras e guardas).
Estruturação dos dados	Atende	ExtractedFact tipado (Pydantic) com normalização por tipo.
Comparação de 2 apólices	Atende	Fluxo A × B com resultado e direção por campo.
Principais diferenças ao usuário	Atende	Tabela campo × resultado + explicação + Relatório D&O.
IA generativa no processamento	Atende	8 tipos de agentes LLM (Gemini/OpenRouter) + embeddings.
Interface de demonstração	Atende	Streamlit + launcher iniciar_app.bat + modo demo offline.

A.3 Boas práticas recomendadas

Boa prática	Como foi atendida
Separar responsabilidades dos componentes	Monólito modular por capacidades de negócio, com camadas domain/application/infrastructure.
Agentes especializados para tarefas distintas	8 tipos de agentes (extração, explicação, pergunta, revisão, extras, seguradora, relatório, fixtures), cada um com prompt versionado.
Documentar a arquitetura	PRD, especificações por módulo, adendos de decisão, handoffs, rastreabilidade de bugs e este relatório.
Justificar decisões técnicas	Seção 13 deste relatório + adendos de decisão versionados no projeto.
Código em módulos reutilizáveis	3 módulos de negócio + kernel compartilhado + composition root.
Mecanismos de tratamento de erros	Erros classificados, <i>retries</i> , timeout de 300s, reexecução e revisão humana.
Ocultar credenciais e chaves de API	Variáveis de ambiente em tempo de execução; verificado que nenhuma chave existe no repositório ou no histórico do Git.
Instruções de instalação e execução	Seção 16 deste relatório + launcher de dois cliques.

Anexo B — Catálogo de campos e vocabulários

O *vertical slice* compara 10 campos críticos por apólice (catálogo fechado), com tipo de valor que dirige a normalização e a regra de comparação. Informações relevantes fora do catálogo são registradas como **achados extras** (exibidos à parte, sem entrar na comparação):

Código	Campo	Tipo	Descrição
limite_agregado	Limite agregado	money	Teto total pago pela seguradora no período de seguro.
limite_por_sinistro	Limite por sinistro	money	Teto pago por evento/sinistro individual.
franquia	Franquia/deducível	money	Valor a cargo do segurado por sinistro antes da cobertura.
vigencia	Vigência	period	Período de cobertura (início e fim) da apólice.
prazo_notificacao_sinistro	Prazo de notificação	number (dias)	Dias após o conhecimento do evento para notificar a seguradora.
extensao_territorial	Extensão territorial	text	Abrangência geográfica da cobertura.
exclusoes_chave	Exclusões-chave	text	Principais exclusões de cobertura listadas na apólice.
limite_defesa_custos	Limite de defesa de custos	money	Teto para custos de defesa judicial/administrativa.
retroatividade	Retroatividade (claims-made)	date	Data a partir da qual fatos geradores são cobertos.
indice_reajuste	Índice de reajuste	number (%)	Percentual de reajuste anual aplicado aos limites.

Vocabulários de resultado: comparação por campo — IGUAL, MAIOR, MENOR, DIVERGENTE, AUSENTE_A, AUSENTE_B, AUSENTES_AMBOS, sempre com direção (A/B) quando aplicável. Relatório D&O — status de cobertura com "Não localizado" (nunca "não existe"), categorias, pesos e cenários de sensibilidade em vocabulário fechado (domain/report_catalog.py).

Anexo C — Fontes de dados e referências

C.1 Fontes de dados utilizadas

- **Apólices reais de seguradoras (versões de teste)** — `tests/apolices/`: AXA D&O, Allianz Condições Gerais (2025 e 2017, esta em imagem/JPG convertida) e Porto D&O, usadas na bancada de validação real (`validate_real.py` , 3 rodadas de 2 arquivos).
- **Apólices sintéticas de teste (produzidas pela equipe)** — duas apólices com estrutura realista de condições gerais/particulares (10 e 9 cláusulas), cobrindo os 10 campos do catálogo, incluindo campo ausente em um dos lados (para validar `AUSENTE_B`). Fixtures em `tests/modules/policy_analysis/fixtures/` .
- **PDFs de teste gerados** — PDFs digitais e um PDF escaneado derivados das fixtures, usados nos testes de integração.
- **Golden set do módulo de avaliação** — `src/modules/evaluation/fixtures/golden_set.json` : valores de referência por campo para medir a acurácia da extração.

Sugestões de fontes públicas para evolução da base: modelos de apólices publicados por seguradoras, documentos disponibilizados pela **SUSEP** (Superintendência de Seguros Privados) e cláusulas padrão do mercado segurador.

C.2 Referências

- GEMINI API — documentação oficial: <https://ai.google.dev/>
- OpenRouter — catálogo de modelos LLM: <https://openrouter.ai/docs>
- Pydantic AI — framework de agentes: <https://ai.pydantic.dev/>
- PaddleOCR / PP-Structure — OCR e análise de layout: <https://github.com/PaddlePaddle/PaddleOCR>
- PyMuPDF — extração e renderização de PDF: <https://pymupdf.readthedocs.io/>
- Qdrant — banco vetorial: <https://qdrant.tech/documentation/>
- DuckDB — banco analítico em arquivo: <https://duckdb.org/docs/>
- Streamlit — interface de aplicações de dados: <https://docs.streamlit.io/>
- SUSEP — Superintendência de Seguros Privados: <https://www.gov.br/susep>